

# auto operator= considered dangerous

**Doc. No.:** P0264R0  
**Date:** 2016-02-11  
**Project:** Programming Language C++  
**Audience:** Evolution Working Group  
**Reply To:** Michael Price  
<[michael.b.price.dev@gmail.com](mailto:michael.b.price.dev@gmail.com)>

## I. Introduction

ISO/IEC 14882:2014 (C++14) introduced the ability to deduce the return type of a function by its definition body. N4309 proposed to extend this further by allowing return type deduction of certain special class member functions that are defined as `=default`. Discussion in EWG was somewhat favorable until the realization that `auto` **never** deduces a reference type, which is the overwhelmingly usual return type for those functions. This paper explores the problem area and offers a small set of alternatives to consider.

## II. Motivation

Paragraph 7.1.6.4, clause 2 of the standard states:

The placeholder type can appear with a function declarator in the *decl-specifier-seq*, *type-specifier-seq*, *conversion-function-id*, or *trailing-return-type*, in any context where such a declarator is valid. If the function declarator includes a *trailing-return-type* (8.3.5), that specifies the declared return type of the function. If the declared return type of the function contains a placeholder type, the return type of the function is deduced from `return` statements in the body of the function, if any.

This wording allows the usage of `auto` or `decltype(auto)` in a function definition, only if there is a trailing return type or a function body that contains at least one `return` statement that can be used to deduce the return type.

```
struct ProbablyWrong
{
    ProbablyWrong() = default;
    ~ProbablyWrong() = default;
    ProbablyWrong(const ProbablyWrong&) { /* ... */ }
    ProbablyWrong(ProbablyWrong&&) = default;
    auto operator=(const ProbablyWrong&) { /* ... */ return *this; }
    auto operator=(ProbablyWrong&&) { /* ... */ return *this; }
};
```

In the above example, the naive "auto everywhere" enthusiast has created a type where both the copy-assignment operator and the move-assignment operator return a copy, instead of the usually desired action of returning `*this` by reference.

## III. Alternative Implementations

There are many different forms that the return type deduction for these functions might reasonably take. Let's explore them. [Note: For brevity's sake, only one method is defined per alternative.]

```
struct CorrectOne
{
    auto& operator=(CorrectOne&&) { /* ... */ return *this; }
};

struct CorrectTwo
{
    decltype(*this) operator=(CorrectTwo&&) { /* ... */ return *this; }
};

struct CorrectThree
{
    decltype(auto) operator=(CorrectThree&&) { /* ... */ return *this; }
};

struct CorrectFour
{
    decltype(auto) operator=(CorrectFour&&) { /* ... */ return *this; }
};

struct CorrectButRisky
{
    decltype(auto) operator=(Correct&& o) { return swap(o); }
```

```
/* POSSIBLY LOTS AND LOTS OF LINES OF CODE */

decltype(auto) swap(Correct&&) { /* ... */ return *this; }

};
```

Most of the variations of return type deduction will do the right thing, but the most "obvious" variation will most likely do something unexpected. Then we have the variation that delegates the return type deduction to an entirely different function. An unsuspecting maintainer could modify the `swap` function without realizing that it will modify the return type of a very important function 100s of lines away.

## IV. What Should We Do?

The committee has several (possibly non-exclusive) options available:

- Exempt special member functions from return type deduction entirely
- Draft special return type deduction rules for special member functions
- Encourage implementations and/or tool vendors to detect such errors
- Do nothing

## V. Acknowledgements

The author would like to thank current and former employers who have supported their work with the committee, Synopsys and Lexmark, respectively. In particular, the keen eyes of EWG members who spotted this bad pattern should be thanked for their diligence.